

State Representation



Dylan Losey

This Lecture



- Why are states so important?
- What is an autoencoder?
- How do we implement autoencoders?



Why are states
so important in
robotics?



States and Control

Minimal states and why they are important for control policies.

- **Setting.** We are learning a control policy $a = \pi_{\theta}(s)$ that maps states s to actions a . This control policy could be learned from an expert dataset (IL) or learned through interacting with the environment (RL).
- **What makes a control policy good?**
 - A good control policy maximizes task performance (minimizes cost) across the test distribution. Ideally, the robot learns this proficient control policy from as little data as possible.
- **What is state compression?**
 - The robot's control policy maps states to actions. Let state $s \in R^K$ be a K-dimensional state. State compression focuses on finding the minimal state (i.e., the state with the fewest dimensions) that the robot should reason over to learn a good control policy.

States and Control

Can we separate controllers and state representations?

- The controller maps input states to output actions. Changing the state representation (e.g., replacing positions with forces) or the dimension of that representation (e.g., adding a new signal) will impact the control law and the resulting actions.

So what work has already been done on state compression?

- State compression is ubiquitous in robot learning. This state compression is commonly achieved through the design of the model architecture (e.g., encoders and decoders) and implicitly encouraged through the loss function (e.g., reconstruction).

States and Control

Why should I care about state compression?

- Or, put another way, why not make observations of the entire world my state? Robots make control decisions based on specific variables. Within an observation, some variables (i.e., features) are relevant, and other variables are irrelevant. In the worst case, the robot learns to make control decisions based on the wrong variables.

Doesn't state compression occur naturally during learning?

- Yes, this does occur to some extent. The robot is learning a control policy to maximize its performance at training time. In developing this policy, the robot implicitly focuses on the variables necessary for decision making (otherwise, it would be unable to achieve high performance during training).

Then why do we need to do anything?

- One option is to rely on the robot to figure it out. The problem with this passive approach is that learning to disentangle the important and irrelevant variables requires additional training data. Moreover, that data needs to have specific properties to facilitate disentanglement.

States and Control

Can you prove that?

- Let's start by talking about how state dimension (K) can affect the amount of training data (N) needed to reach a given level of performance. Consider a simple linear regression problem. The robot linearly maps its state s to control action a using parameters θ . At training time, the robot has access to a set of N independent state-action pairs that are sampled according to:

$$a = s^T \theta^* + \epsilon, \quad \epsilon \sim N(0, \sigma_\epsilon^2), \quad s \sim N(0, \Sigma_s)$$

States and Control

The true weights θ^* are unknown. The robot estimates these weights as θ using linear regression. We are interested in the error between the control actions the robot thinks it should take, and the correct control actions:

$$e = (a - \hat{a}) = s^T (\theta^* - \theta)$$

For now, let's say that at test time the robot encounters states x sampled from the same distribution as at training time. In this case, the unbiased error is a normal distribution:

$$e \sim N\left(0, \frac{K\sigma_\epsilon^2}{N}\right)$$

States and Control

For this example, we conclude that:

- Increasing the dimension of the state (K) linearly *increases* the error variance.
- Increasing the number of datapoints (N) linearly *decreases* the error variance.
- Increasing the amount of variance in the demonstrated state-action pairs (σ_ϵ^2) linearly *increases* the error variance.

Put another way, if we double the state dimension, then we need twice as much training data to reach the same level of performance!

$$e \sim N\left(0, \frac{K\sigma_\epsilon^2}{N}\right)$$

States and Control

You previously mentioned the data needs specific properties. What do you mean?

- Let's stick with the linear regression problem setting, but now consider how the learned control policy generalizes. Previously the robot encountered states x sampled from the same distribution at training and test time. Now let's say the robot encounters states sampled from the standard normal distribution at test time:

$$s_{train} \sim N(0, \Sigma_s), \quad s_{test} \sim N(0, I_K)$$

- In this case, the error between the correct control actions and the control actions learned using linear regression becomes:

$$e \sim N\left(0, \frac{\sigma_\epsilon^2}{N} \cdot \text{tr}(\Sigma_s^{-1})\right)$$

States and Control

To provide a bit more intuition on this result, consider the upper bound on error variance:

$$\frac{\sigma_{\epsilon}^2}{N} \cdot \text{tr}(\Sigma_s^{-1}) \leq \frac{K \sigma_{\epsilon}^2}{N \lambda_{\min}(\Sigma_s)}$$

where $\lambda_{\min}$ is the smallest eigenvalue of Σ_s . With this example in mind, we can go back to the original question. Specifically focusing on the state, we see that — if the states observed at training time are correlated, but the states encountered at test time are not, then the error variance increases. In the worst case, two states are identical at training time but different at test time; in this case, the upper bound shown above goes to infinity!

States and Control

So what does this all mean for state compression?

- The robot can learn the correct control policy on its own, without the designer providing a minimal state. But doing so requires additional training data. At least for linear regression, the amount of data required to reach a given performance increases at the same rate as the state dimension. The amount of data increases even more if we want that control policy to generalize across state distributions. For instance, if multiple states are correlated at training time — and then uncorrelated at test time — the robot needs additional examples to disentangle which states are relevant for choosing the correct control actions.

Then the advantage of state compression is faster learning?

- Sort of. If we can remove unnecessary variables from the state, the robot can learn the correct control policy from less training data. But if we can remove unnecessary variables that are spuriously correlated to important variables, then we also enhance policy generalization. So it's more accurate to say the advantage of state compression is that (a) we learn a more general control policy (b) based on fewer data points.



OK. You've
convinced me that
state compression
is good. But how
do we do it?



Review

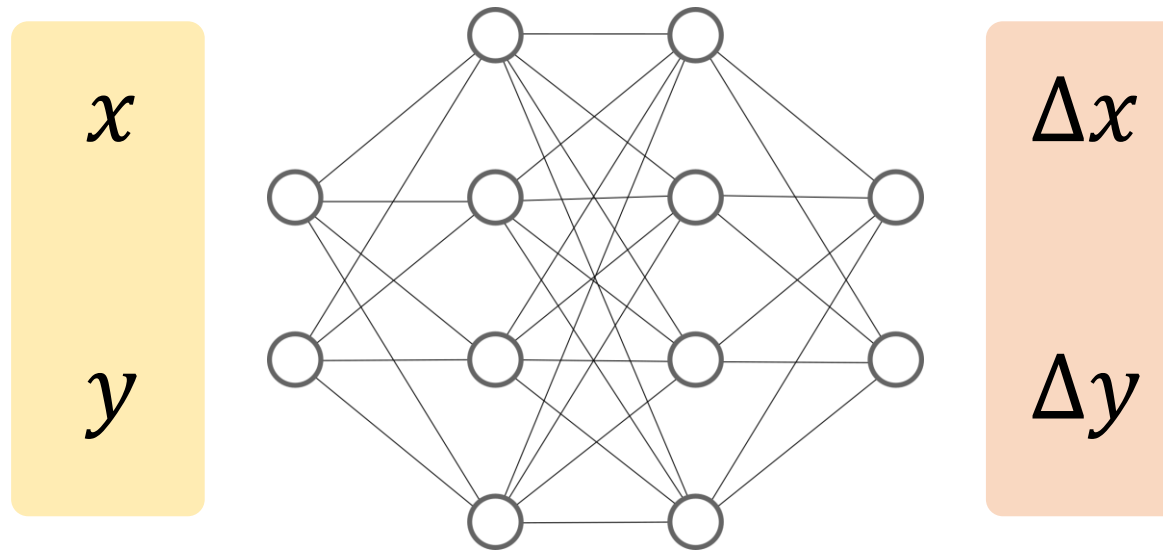
Let's start with **behavior cloning**.

- Given a dataset of states and actions, we learned a mapping from s to a

Review

Let's start with **behavior cloning**.

- Given a dataset of states and actions, we learned a mapping from s to a



Review

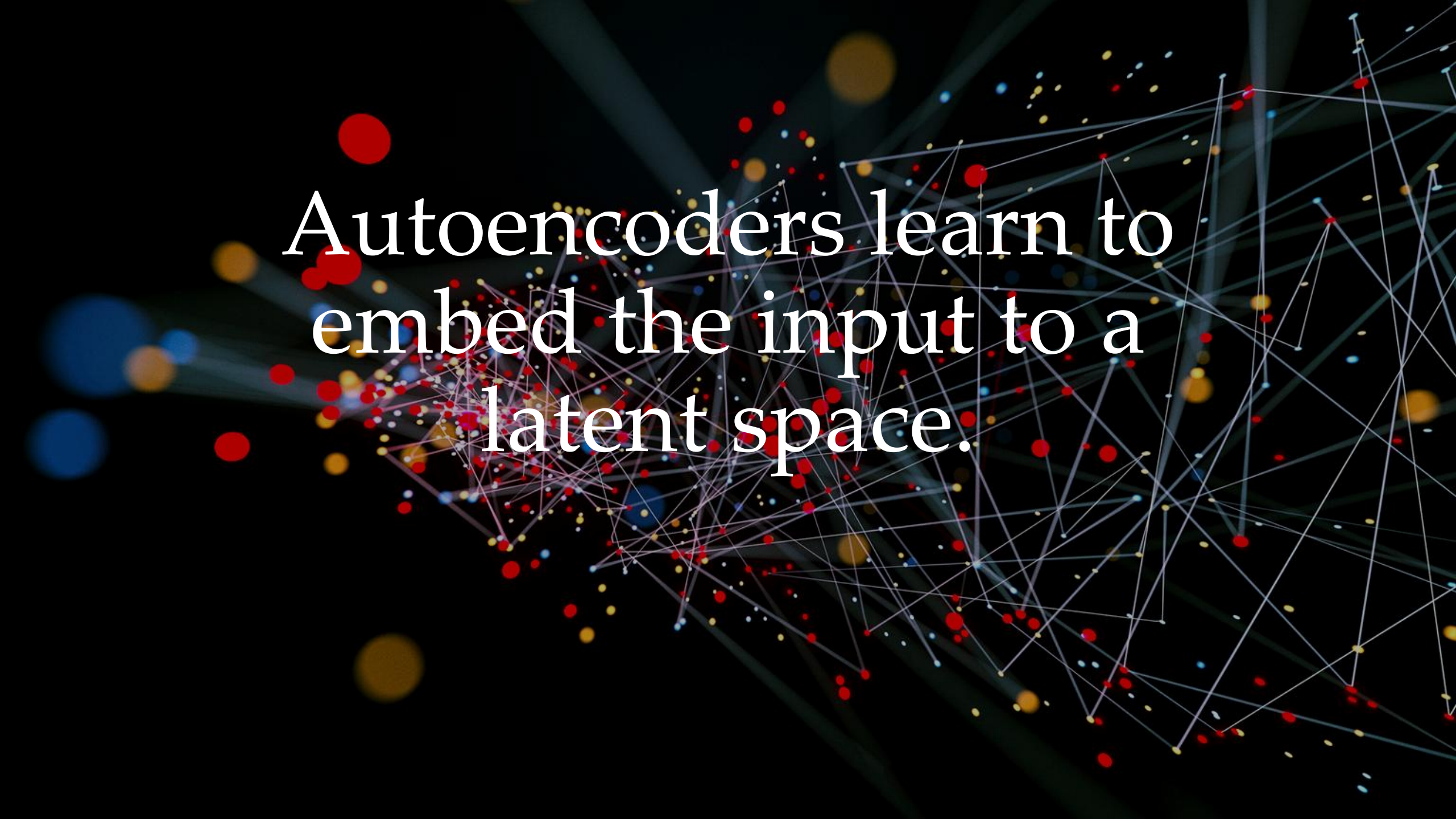
Let's start with **behavior cloning**.

- Given a dataset of states and actions, we learned a mapping from s to a
- We trained the model to minimize the loss function:

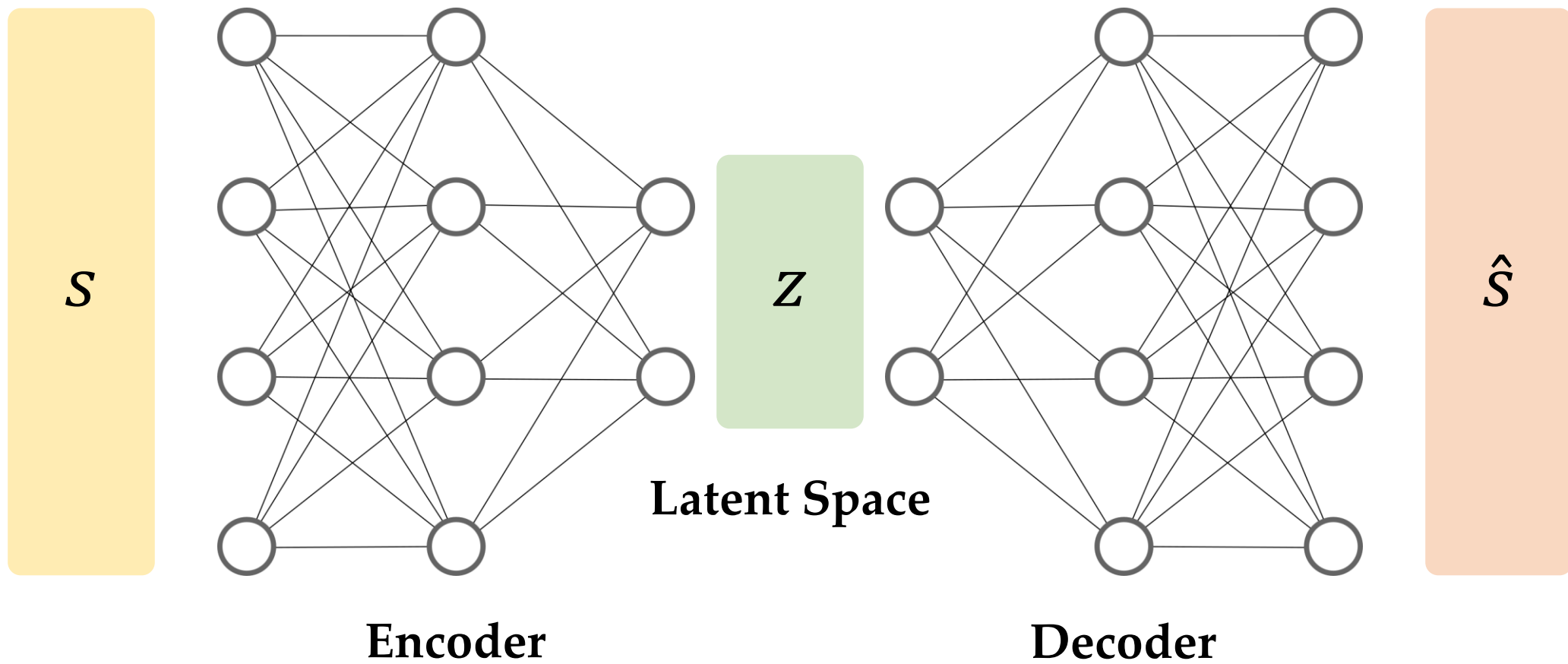
$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{(s, a_h) \in D} \|a_h - \pi_{\theta}(s)\|^2$$

Loss depends on the
model weights θ

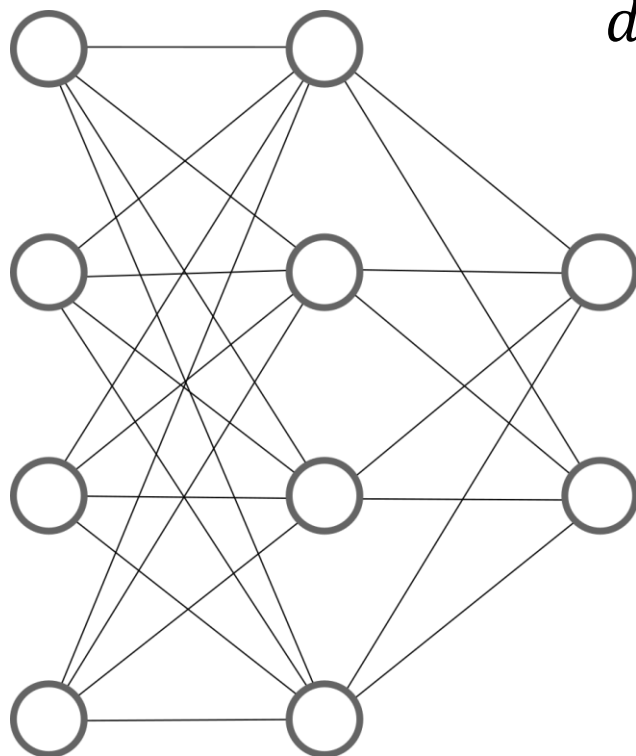
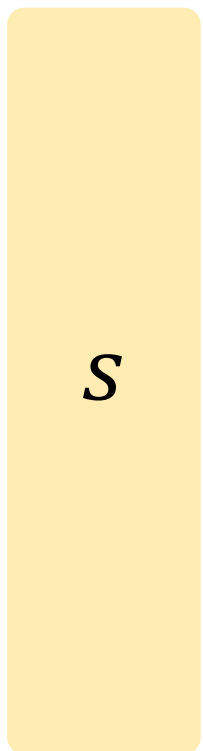
Error between actual and predicted

The background is a dark, textured surface. It features a complex network of thin, light-colored lines connecting numerous small, colored dots (nodes) in red, blue, and yellow. These dots are scattered across the frame, with a higher concentration in the center-left area. Additionally, there are several large, out-of-focus circular light spots in shades of red, blue, and yellow, creating a bokeh effect. The overall aesthetic is high-tech and abstract, suggesting a digital or data-driven environment.

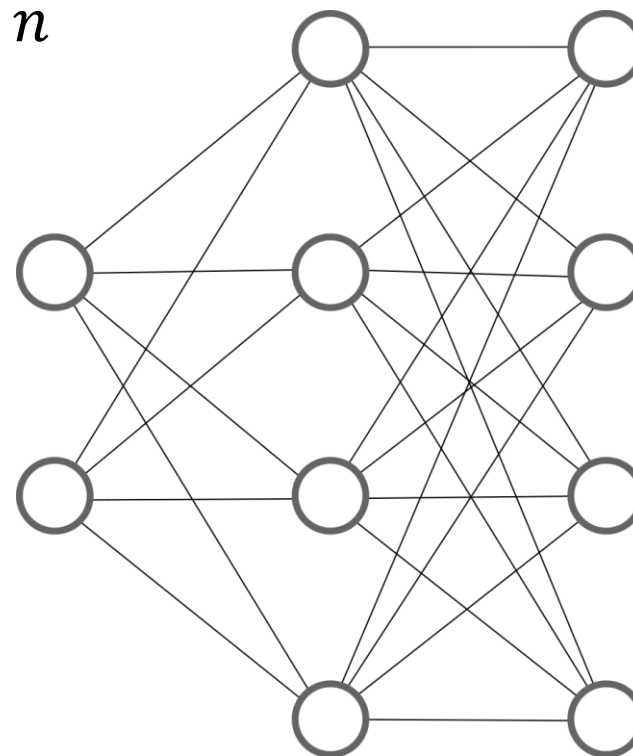
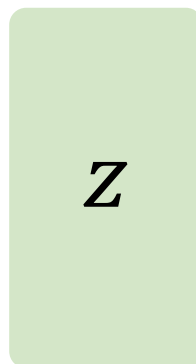
Autoencoders learn to
embed the input to a
latent space.



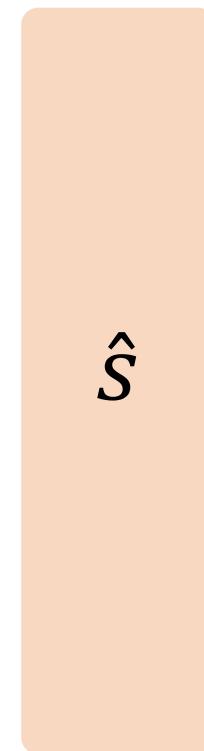
$$\dim(s) = m$$



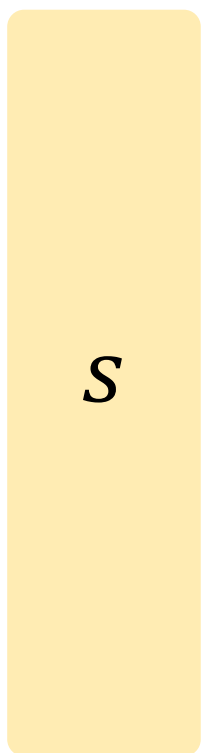
$$\dim(z) = n$$
$$n < m$$



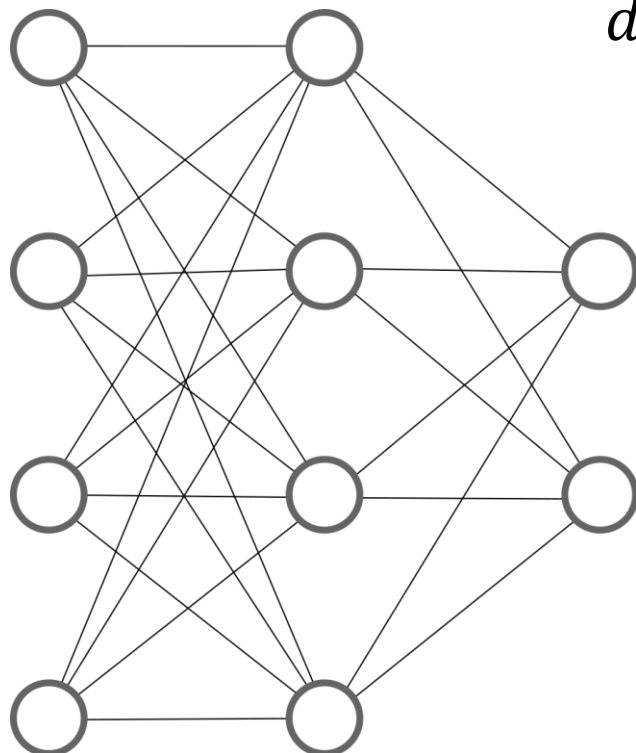
$$\dim(\hat{s}) = m$$



$$\dim(s) = m$$

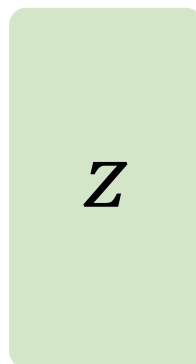


S

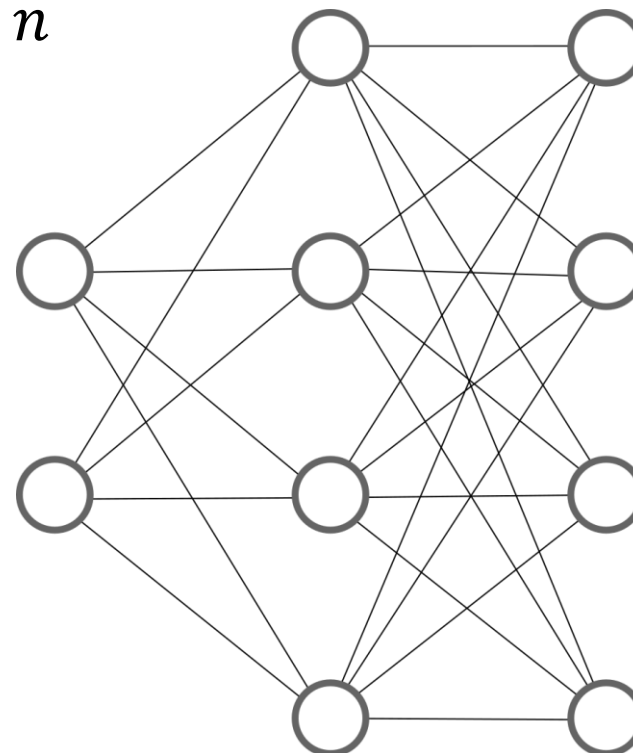


$$\dim(z) = n$$

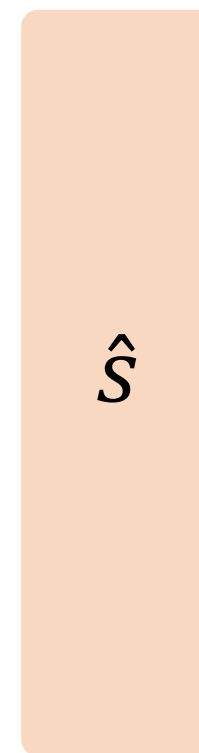
$$n < m$$



Z



$$\dim(\hat{s}) = m$$



$\hat{S}$

$z \in Z$ is the latent variable. Z is the latent space (continuous).

Autoencoders

Autoencoders learn to embed the input to a **latent space**.

Put another way, autoencoders learn a lower-dimensional representation of the input that is *sufficient* to capture that input.

Autoencoders contain two models:

- Encoder that encodes $\varphi(s) \rightarrow z$
- Decoder that decodes $\phi(z) \rightarrow \hat{s}$

Autoencoders

Autoencoders contain two models:

- Encoder that encodes $\varphi(s) \rightarrow z$
- Decoder that decodes $\phi(z) \rightarrow \hat{s}$

We train the models to *accurately reconstruct* the input data.

$$\hat{s} = \phi(\varphi(s))$$

Combining the encoder and decoder

Autoencoders

Autoencoders contain two models:

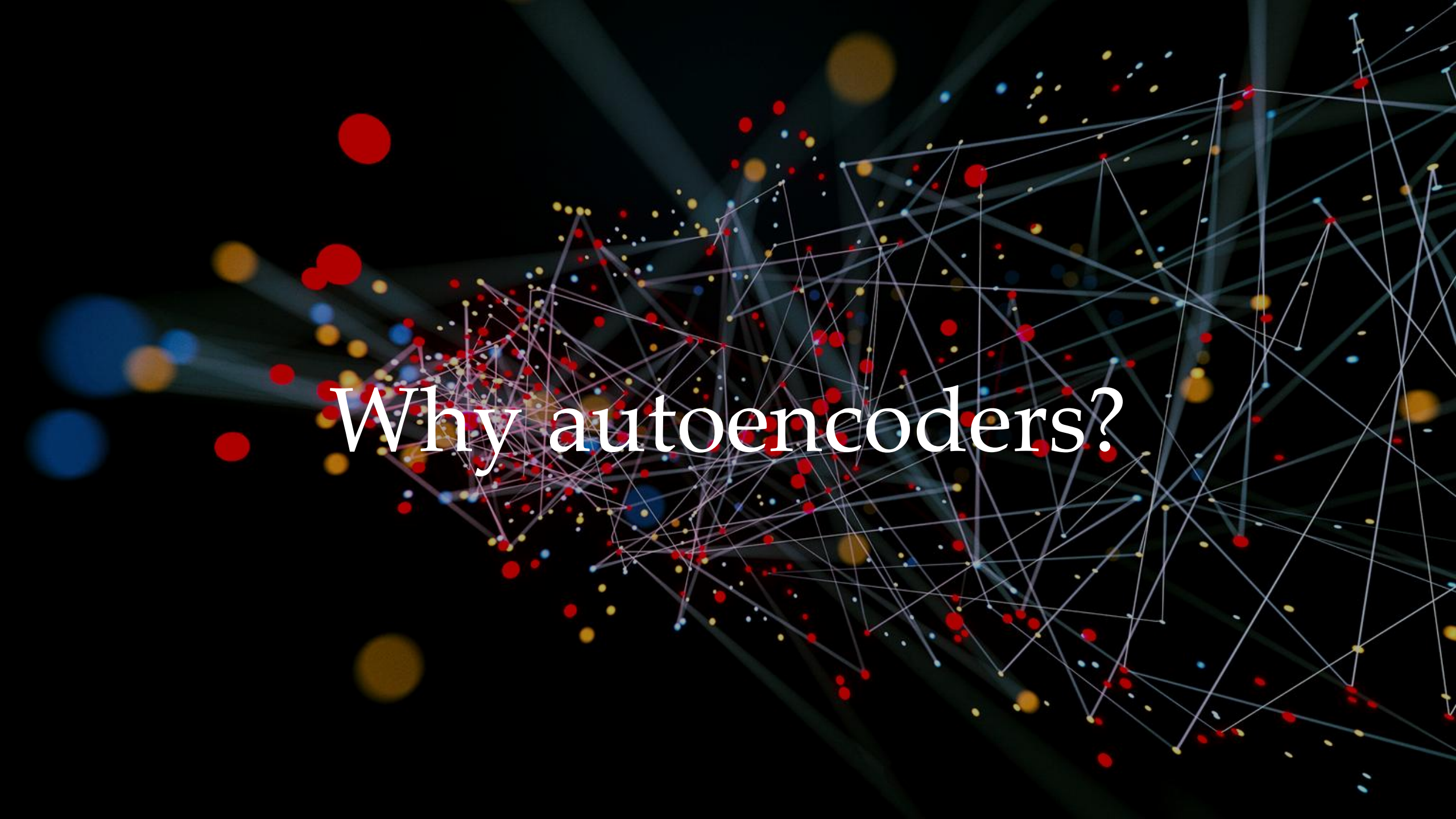
- Encoder that encodes $\varphi(s) \rightarrow z$
- Decoder that decodes $\phi(z) \rightarrow \hat{s}$

We train the models to *accurately reconstruct* the input data.

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{s \in D} \|s - \phi(\varphi(s))\|^2$$

Models have weights θ

Error between actual and predicted

The background is a dark, textured surface. It features a complex network of thin, light-colored lines connecting numerous small, colored dots (nodes) in red, yellow, and blue. These nodes are scattered across the frame, with a higher concentration in the center-left area. Additionally, there are several large, out-of-focus circular light spots in red, yellow, and blue, creating a bokeh effect. The overall aesthetic is high-tech and abstract, suggesting themes of data, networks, or artificial intelligence.

Why autoencoders?

Learning Latent Spaces

$$\varphi(s) \rightarrow z$$

Once trained, we can use autoencoders to **embed** high-dimensional signals to low-dimensional signals. Consider embedding a large video file!

$$\phi(z) \rightarrow \hat{s}$$

We can also sample latent variables z and decode them to **reconstruct** high-dimensional states. For example, a new human face!

Low-Dimensional Control

- Consider a robot arm with state s and action a
- The human provides low-dimensional input z , where $z < a$

We want to learn a mapping from state and input to action:

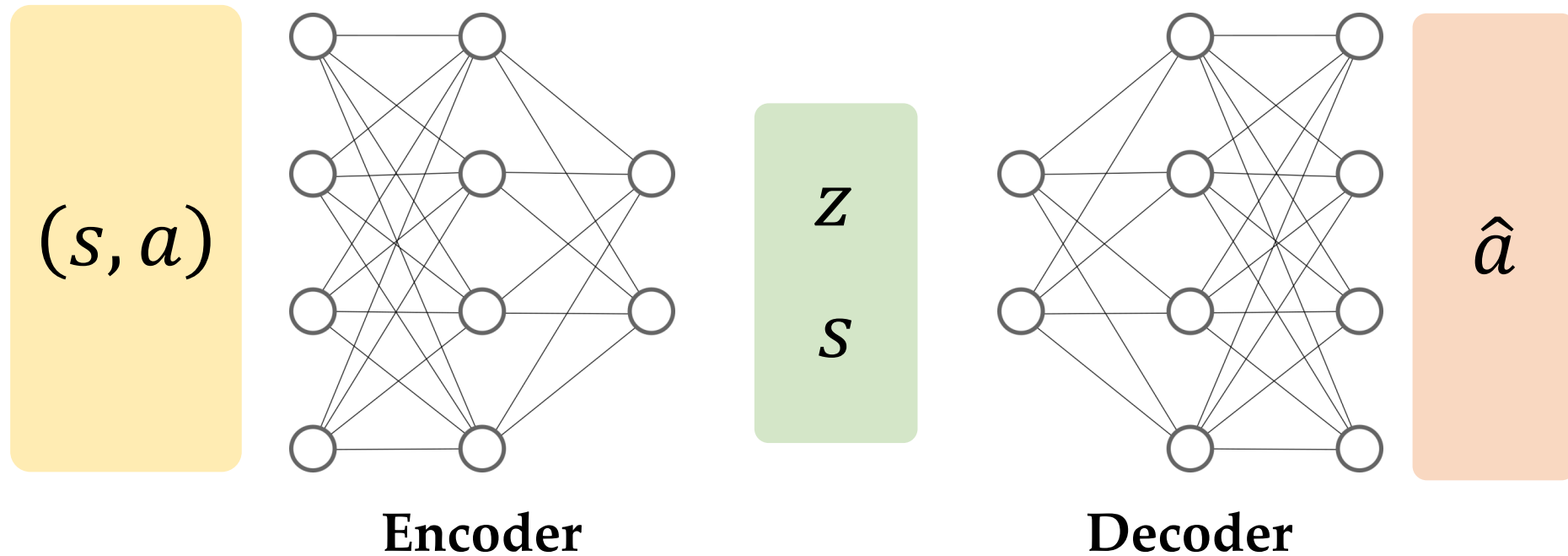
$$\phi(s, z) \rightarrow a$$

Learning Latent Actions



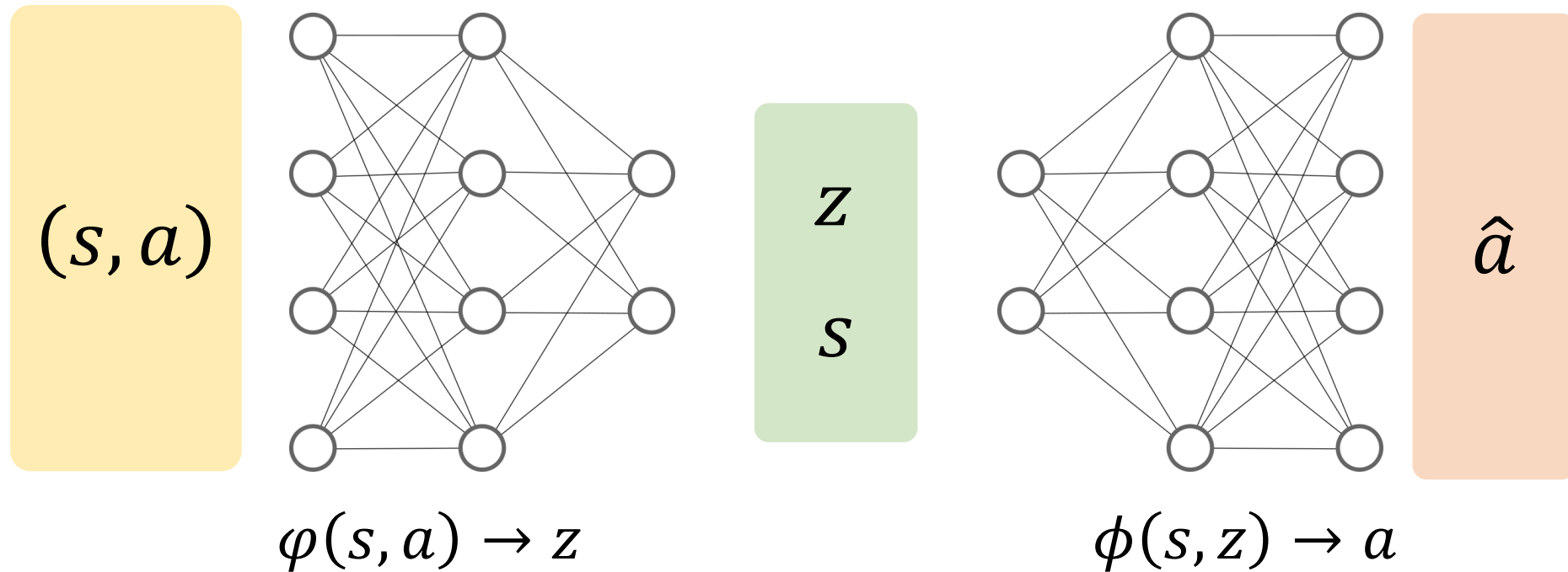
Latent Actions

To learn the mapping $\phi(s, z) \rightarrow a$ we will use a **conditional autoencoder**.



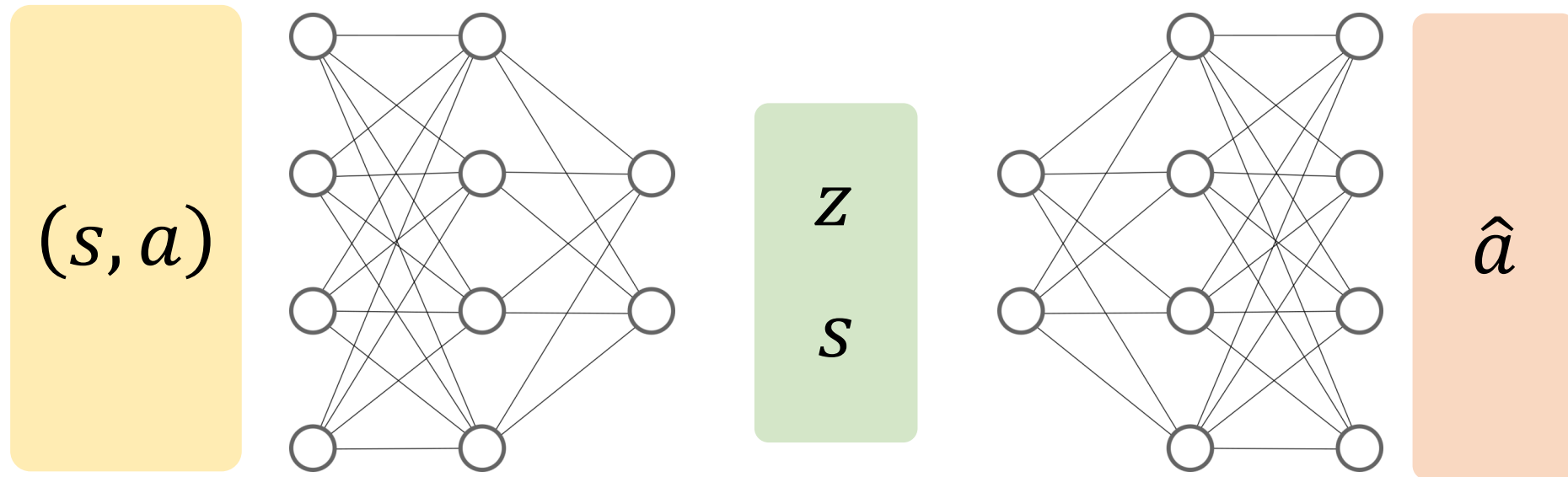
Latent Actions

To learn the mapping $\phi(s, z) \rightarrow a$ we will use a **conditional autoencoder**.



Latent Actions

To learn the mapping $\phi(s, z) \rightarrow a$ we will use a **conditional autoencoder**.



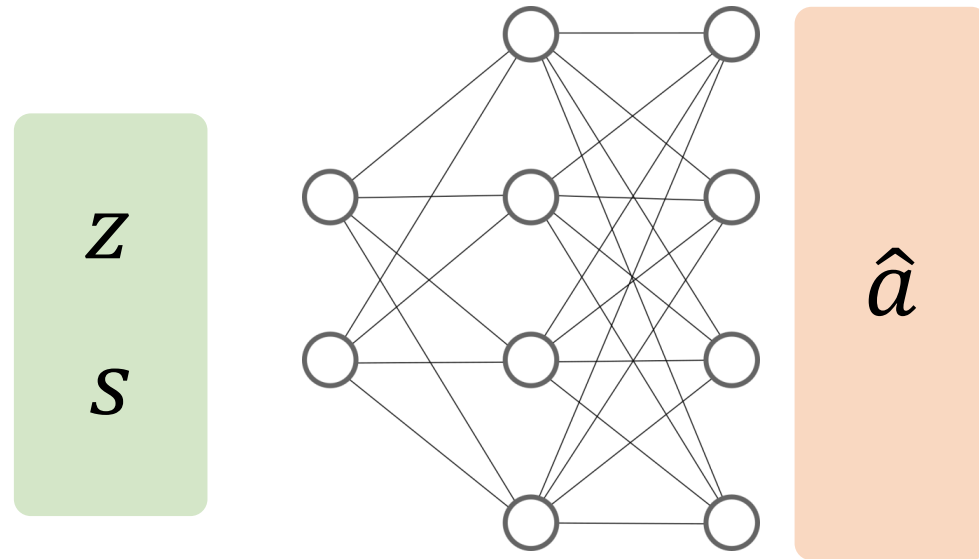
$$\phi(s, a) \rightarrow z$$

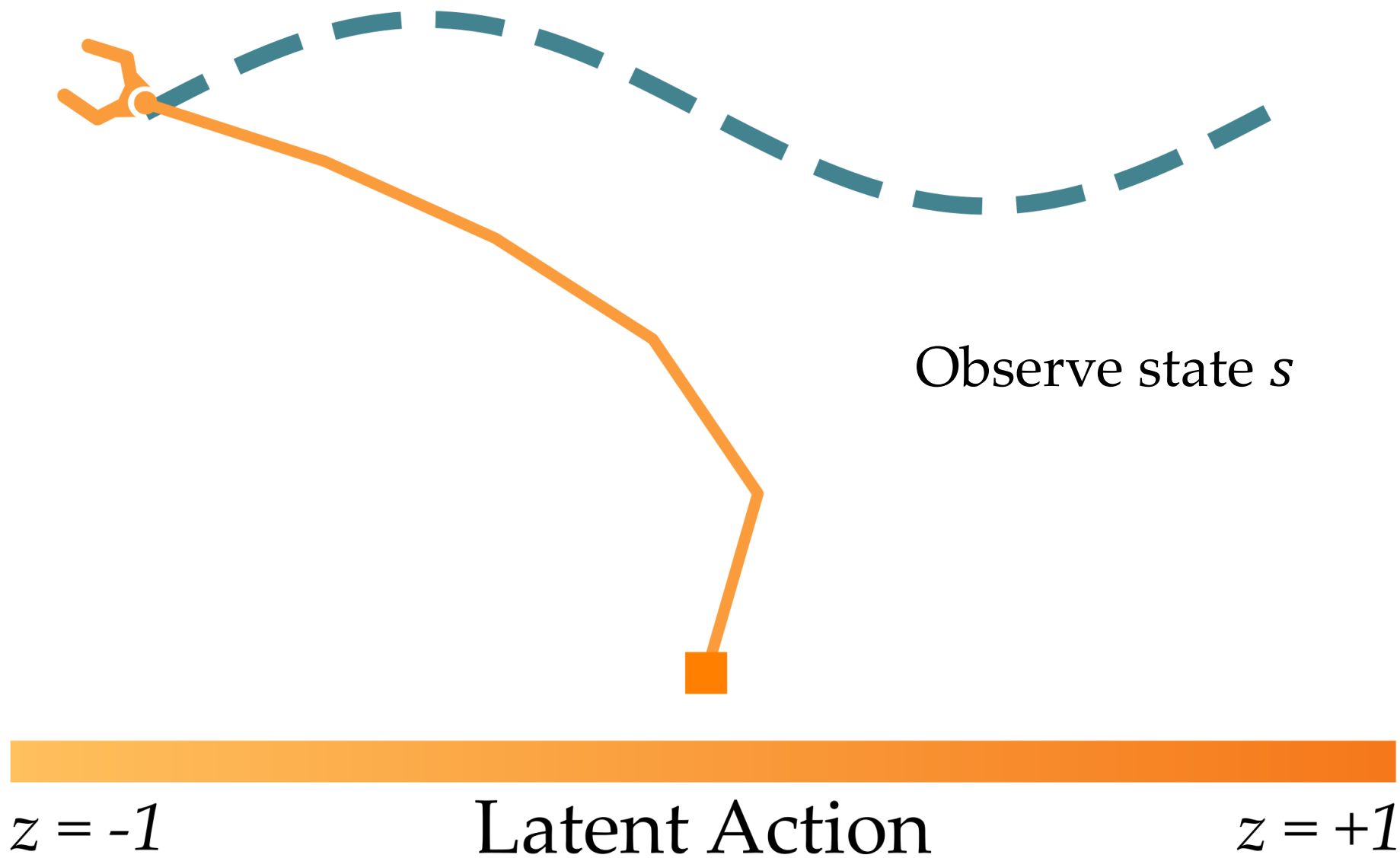
The decoder is *conditioned* on state s . Take s from input and pass directly to decoder.

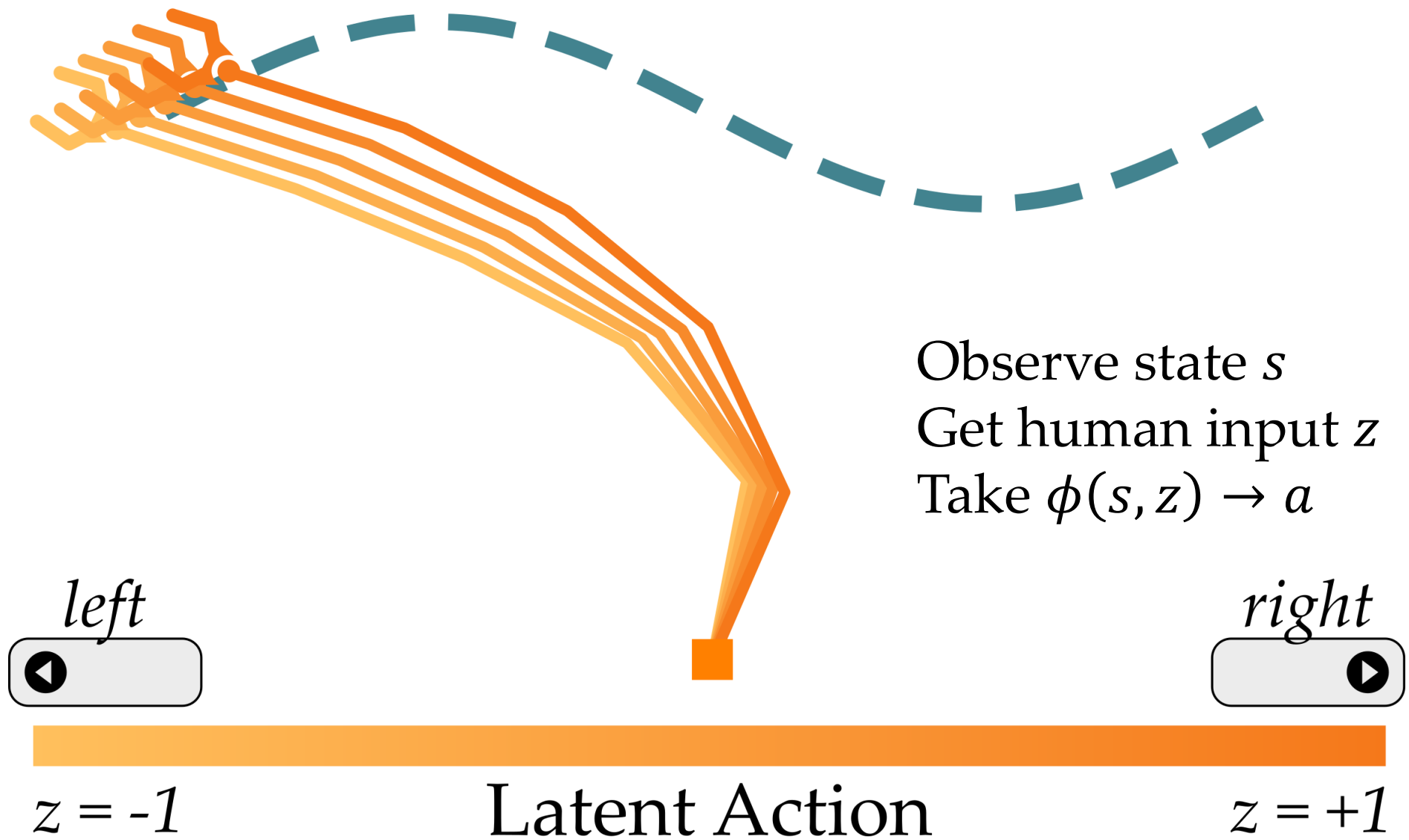
Latent Actions

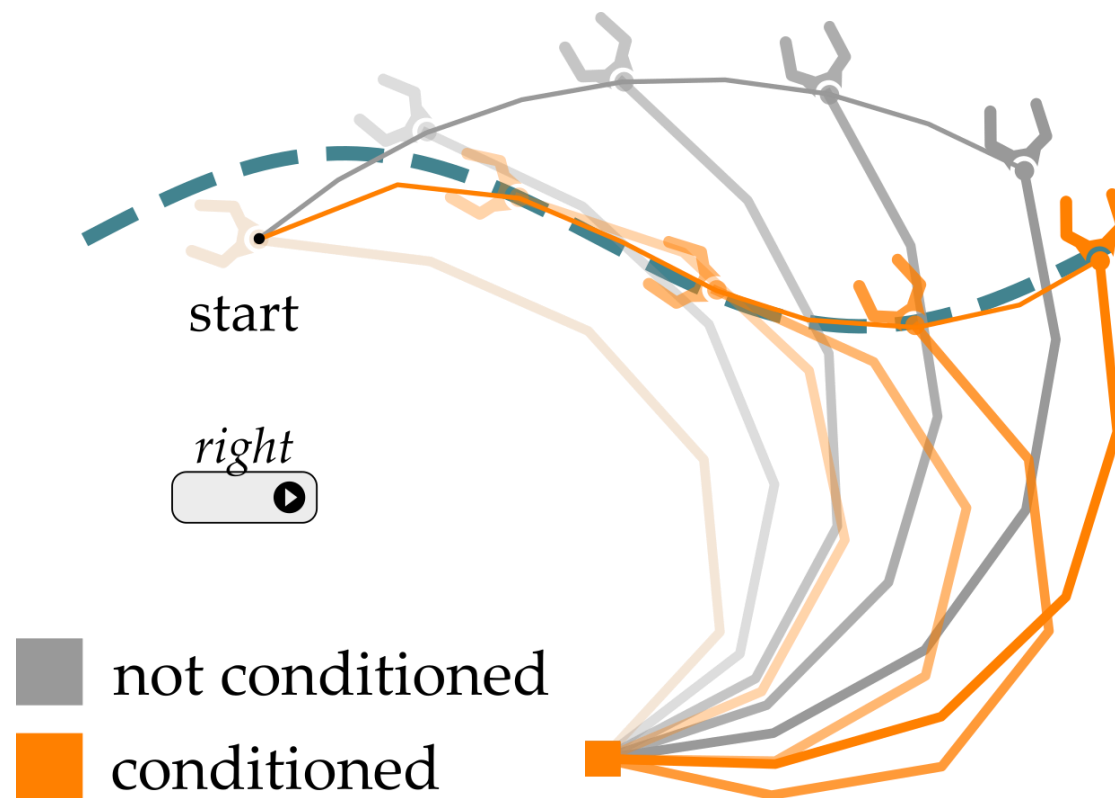
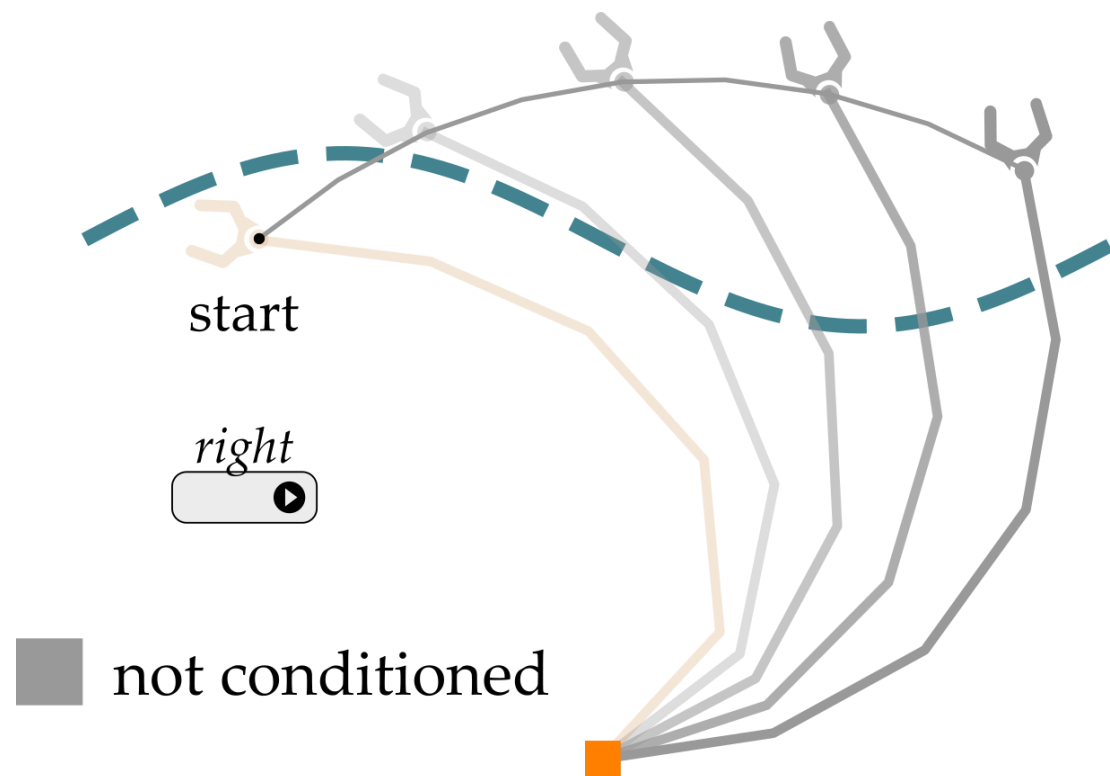
To learn the mapping $\phi(s, z) \rightarrow a$ we will use a **conditional autoencoder**.

Once trained, we control the robot using only the decoder to get action a









Latent Actions

Offline collect dataset $D = \{(s^1, a^1), \dots, (s^N, a^N)\}$

Initialize models:

- Encoder $\varphi(s, a) \rightarrow z$
- Decoder $\phi(s, z) \rightarrow a$

Train encoder and decoder to minimize loss:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{(s,a) \in D} \|a - \phi(s, \varphi(s, a))\|^2$$

Models have weights θ

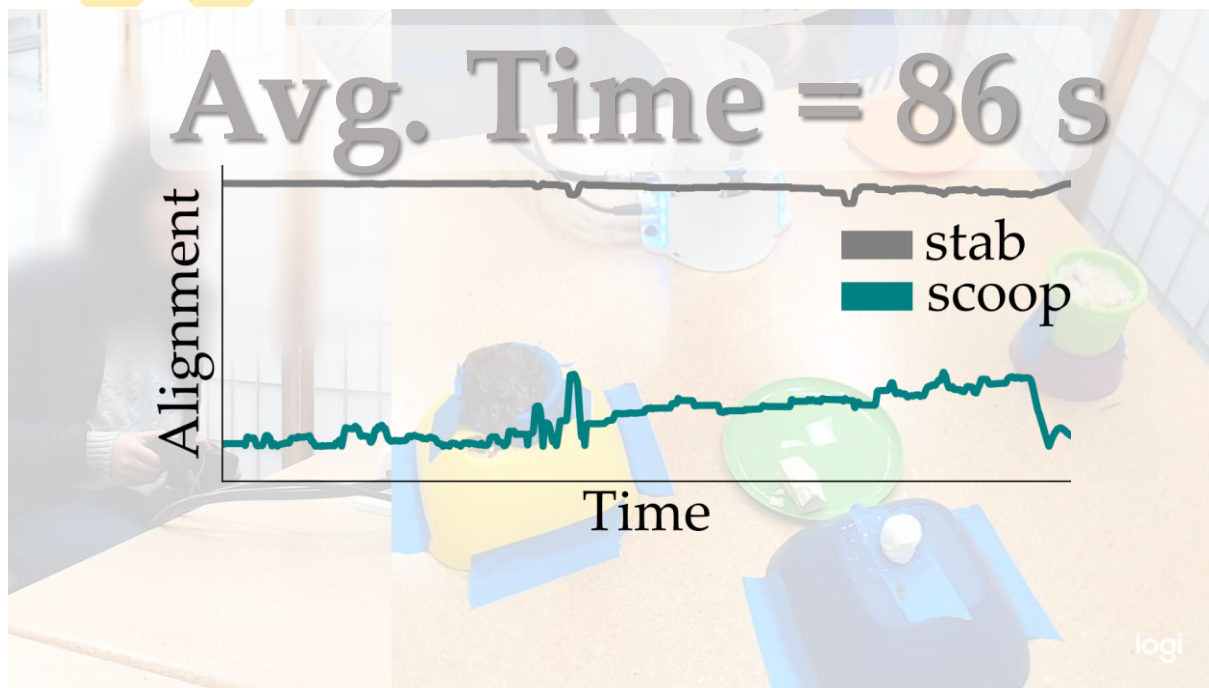
Error between actual and predicted

Latent Actions

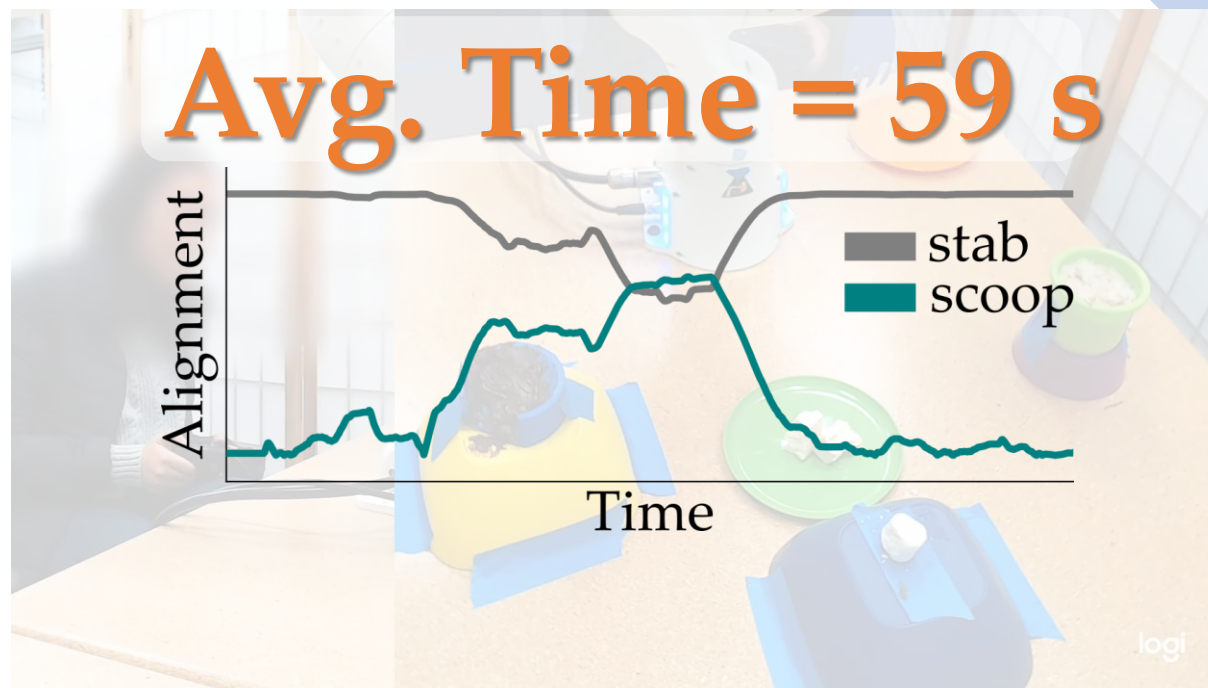
Online take trained decoder $\phi(s, z) \rightarrow a$

At each timestep t :

- Observe state s and human input z
- Get decoded action $\phi(s, z) \rightarrow a$
- [*Optional*] Treat decoded action as a_h
- [*Optional*] Use predict and blend to assist human
- Transition to next state



R



LA + SA (ours)

This Lecture



- Why are states so important?
- What is an autoencoder?
- How do we implement autoencoders?

Next Lecture



- Using game theory to frame human-robot interaction